

v1.7 — Case Management, Workflow, Tasks, Deadlines & Legal Process Specification

Great. **v1.7** moves from the document/records foundation into the operational heart of ILCMS: how a legal matter is opened, assigned, worked, monitored, progressed, completed, and eventually closed.

ILCMS — Icelandic Legal Case Management System

Technical Specification v1.7

Case Management, Workflow, Tasks, Deadlines & Legal Process Specification

Project: Icelandic Legal Case Management System (ILCMS)

Version: 1.7

Status: Technical Specification

Target Market: Iceland

Primary Language: Icelandic

Backend: Python / FastAPI

Database: PostgreSQL

Frontend: Web application

Deployment: Kubernetes / private cloud / on-premises / air-gapped capable

1. Purpose

This specification defines the core case-management and legal-workflow functionality of ILCMS.

The system shall support the lifecycle of a legal matter from initial intake through:

Intake

↓

Case Opening

↓

Classification

↓

Assignment

↓

Investigation / Preparation

↓

Legal Work

↓

Court / External Process

↓

Resolution

↓

Closing

↓

Retention / Archiving

The system shall provide structured support for:

- cases;
- parties;
- lawyers;
- teams;
- assignments;
- tasks;
- deadlines;
- hearings;
- court events;
- correspondence;
- notes;
- workflow;
- reminders;
- escalations;
- case status;
- case closure;
- workload management;
- auditability.

The system must remain configurable because legal procedures differ between case types, organizations and courts.

2. Design Philosophy

ILCMS is a **case-management system**, not an autonomous legal decision-making system.

The system may:

- organize information;

- calculate dates;
- generate reminders;
- recommend workflow actions;
- identify missing information;
- generate administrative notifications;
- assist with document preparation.

It must not silently make legal decisions on behalf of the responsible lawyer.

For example:

System:

"Deadline occurs in 3 days."

Correct.

System:

"Therefore the client should definitely file this motion."

Not an automatic legal decision.

3. Case as the Central Domain Object

The case is the primary operational object.

Case

|

+-- Parties

|

+-- Lawyers

|

+-- Teams

|

+-- Documents

|

+-- Tasks

|

+-- Deadlines

|

+-- Hearings

|

+-- Correspondence

|

+-- Notes
|
+-- Events
|
+-- Expenses
|
+-- Time entries
|
+-- Court proceedings
|
+-- Audit history

The case should provide a coherent view of all related activities.

4. Case Lifecycle

The standard lifecycle should support:

INTAKE
↓
OPEN
↓
ACTIVE
↓
PENDING
↓
RESOLVED
↓
CLOSED
↓
ARCHIVED

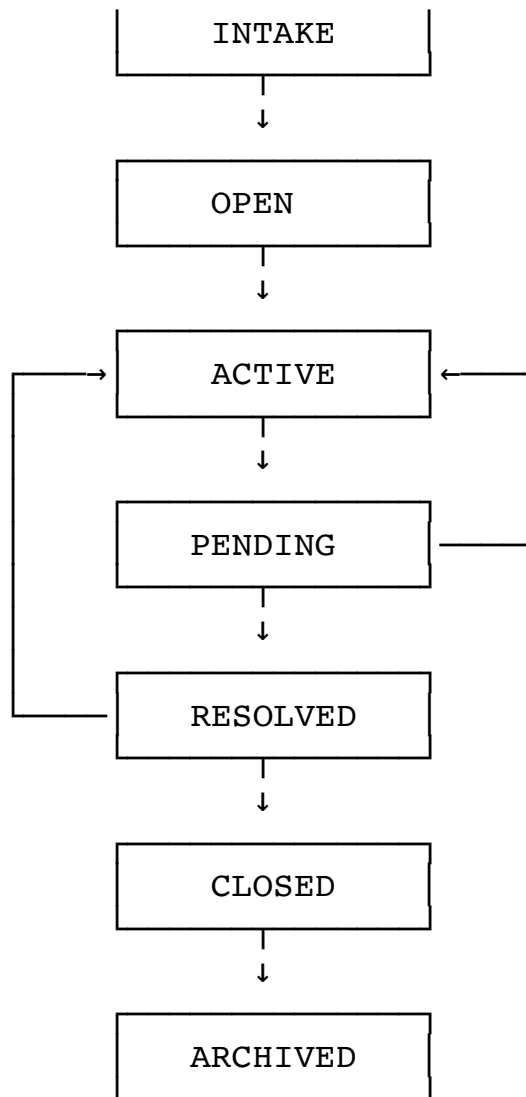
Additional states may include:

REJECTED
CONFLICT_CHECK
ON_HOLD
SUSPENDED
REOPENED

The state machine must be configurable.

5. Case State Machine

Example:



Not every transition should be available to every role.

6. Case Creation

Creating a case should require at minimum:

Case title
Case type
Organization
Opening date
Primary party/client
Responsible lawyer or team
Classification
Depending on case type:

- opposing party;
- court;

- court case number;
- jurisdiction;
- matter description;
- source/referral;
- priority.

7. Case Number

ILCMS shall generate a unique case number.

Example:

M-2026-00125

The format must be configurable.

A case number should be:

- human-readable;
- unique within the appropriate organization;
- immutable once assigned.

The UUID remains the technical identifier.

8. Case Title

The case title should be explicitly stored.

Example:

Stefán Jónsson gegn Norðurlandi ehf.

The system should not rely on dynamically constructing the title from party names.

Party names can change or be corrected.

9. Case Type

Case types shall be configurable.

Possible initial values:

CIVIL

CRIMINAL
FAMILY
EMPLOYMENT
CORPORATE
CONTRACT
PROPERTY
INSURANCE
IMMIGRATION
ADMINISTRATIVE
DEBT
ESTATE
OTHER

Organizations may define additional types.

10. Case Classification

Each case shall have a classification.

Example:

```
Civil
└─ Contract
    └─ Commercial contract
```

Classification should support the records-management requirements defined in v1.6.

11. Case Priority

Cases may have a priority:

LOW
NORMAL
HIGH
CRITICAL

Priority is an operational value.

It should not automatically imply legal urgency unless explicitly configured.

12. Case Confidentiality

Cases should support configurable confidentiality levels:

NORMAL
CONFIDENTIAL

HIGHLY_CONFIDENTIAL
RESTRICTED

Effective access is determined by the authorization architecture from v1.5.

13. Case Owner

Every active case should have a responsible owner.

Example:

Case

```
|  
+-- Responsible Lawyer  
+-- Case Manager  
+-- Team
```

The owner is accountable for case administration but does not automatically receive permissions beyond their assigned role.

14. Case Assignment

Cases may be assigned to:

- individual lawyers;
- assistants;
- case managers;
- teams;
- departments.

Assignment should be represented explicitly.

Case

```
|  
+-- Owner  
+-- Assigned Users  
+-- Assigned Teams
```

15. Assignment History

Assignments must be auditable.

Example:

01.09.2026

Assigned to Lawyer A

05.09.2026

Transferred to Lawyer B

10.09.2026

Added Assistant C

Historical assignments should not be overwritten.

16. Team-Based Access

Teams may be used to simplify case access.

Example:

Corporate Law Team

|

+-- Lawyer A

+-- Lawyer B

+-- Assistant C

Assigning a case to the team may grant access according to configurable team permissions.

17. Organization Structure

Organizations may optionally contain:

Organization

|

+-- Departments

|

+-- Teams

|

+-- Users

This structure can support:

- workload management;
- assignment;
- reporting;
- access control.

18. Party Management

Cases may contain multiple parties.

Example:

Case

```
|
+-- Client
+-- Opposing Party
+-- Witness
+-- Court
+-- Other
```

Party roles must be explicitly represented.

19. Party Roles

Initial roles may include:

```
CLIENT
OPPOSING_PARTY
WITNESS
COUNSEL
COURT
AUTHORITY
THIRD_PARTY
OTHER
```

The same party may have different roles in different cases.

20. Party Relationships

Party relationships should be modeled separately from the party itself.

Example:

Case A

```
|
+-- Jón Jónsson
    role = CLIENT
```

Case B

```
|
+-- Jón Jónsson
```

`role = WITNESS`

The global party record must not permanently assign a single role.

21. Conflict Checking

Before opening a case, the system should support conflict checking.

A conflict check may search:

- client;
- opposing party;
- related organizations;
- known aliases;
- relevant persons/entities.

The result should be:

`NO_CONFLICT_FOUND`
`POTENTIAL_CONFLICT`
`CONFLICT_CONFIRMED`
`REVIEW_REQUIRED`

The system should identify potential conflicts; final legal determination remains a human responsibility.

22. Conflict Check Audit

A conflict check should record:

`performed_at`
`performed_by`
`search_terms / parties`
`result`
`reviewer`
`decision`

Sensitive search information should be protected.

23. Case Intake

Case intake should support an initial workflow:

`New inquiry`

↓
Capture client
↓
Conflict check
↓
Initial assessment
↓
Accept / Reject
↓
Open case

24. Intake Record

An intake record may contain:

intake_id
received_at
source
prospective_client
matter_description
assigned_reviewer
conflict_status
intake_status
decision
decision_at

Intake data should be retained according to configured policy.

25. Intake Sources

Possible sources:

EMAIL
PHONE
WEB_FORM
REFERRAL
EXISTING_CLIENT
COURT
INTERNAL
OTHER

26. Case Workflow

A workflow defines the expected progression of a case.

Example:

Civil Case Workflow

Intake

↓

Conflict Check

↓

Open Case

↓

Initial Review

↓

Collect Evidence

↓

Prepare Claim

↓

Submit

↓

Await Response

↓

Prepare Hearing

↓

Hearing

↓

Decision

↓

Close

Workflows should be configurable.

27. Workflow Templates

A workflow template may define:

workflow_id

name

case_type

version

states

transitions

task_templates

deadline_templates

notifications

required_documents

Templates should be versioned.

28. Workflow Versioning

When a workflow changes:

Workflow v1

Workflow v2

existing cases should normally remain associated with the version under which they were started unless explicitly migrated.

This avoids unexpectedly changing active cases.

29. Workflow State

A case should contain its current workflow state separately from its overall case status.

Example:

```
case_status = ACTIVE
```

```
workflow:
```

```
    CIVIL_LITIGATION_V3
```

```
workflow_state:
```

```
    AWAITING_DEFENDANT_RESPONSE
```

This provides more precise operational information.

30. State Transitions

Transitions should be explicit.

Example:

```
ACTIVE
```

```
|
```

```
|
```

```
submit_response
```

```
v
```

```
AWAITING_RESPONSE
```

The API should reject invalid transitions.

31. Transition Authorization

A user must have permission to perform a transition.

For example:

```
case:close
case:reopen
case:assign
must be independently authorized.
```

32. Transition Validation

Some transitions may require conditions.

Example:

```
CLOSE CASE
|
+-- outstanding mandatory tasks?
+-- unresolved deadlines?
+-- active legal hold?
+-- required documents?
```

If policy requires unresolved items to be addressed, the transition must be blocked.

The conditions must be configurable.

33. Tasks

Tasks represent work that must be performed.

A task should contain:

```
task_id
case_id
title
description
status
priority
assigned_to
assigned_team
created_at
due_date
completed_at
created_by
completed_by
```

34. Task Status

Recommended:

TODO

IN_PROGRESS

BLOCKED

COMPLETED

CANCELLED

35. Task Priority

LOW

NORMAL

HIGH

URGENT

Task priority is operational and does not automatically change legal deadlines.

36. Task Assignment

A task may be assigned to:

- user;
- team;
- department.

Only one user should normally be the directly responsible assignee at a time.

Team assignments may optionally support claiming.

37. Task Dependencies

Tasks may depend on other tasks.

Example:

Collect evidence

↓

Review evidence

↓

Prepare pleading

↓

Lawyer approval

↓

Submit

The system should support dependency relationships.

38. Task Dependency Types

Initial support:

BLOCKS
DEPENDS_ON
RELATED_TO
Example:

Task B DEPENDS_ON Task A

39. Recurring Tasks

The system may support recurring tasks.

Examples:

- monthly reporting;
- recurring client updates;
- periodic compliance checks.

Recurring task generation must be deterministic and idempotent.

40. Task Templates

Workflow templates may automatically create tasks.

Example:

```
Case opened
|
+-- Create conflict-review task
+-- Create client-identification task
+-- Create initial assessment task
```

41. Task Completion

Completing a task should record:

completed_at
completed_by

completion_note

The original task history must remain auditable.

42. Task Reopening

Completed tasks may be reopened if permitted.

The system should preserve the history:

TODO

↓

COMPLETED

↓

REOPENED

↓

COMPLETED

43. Deadlines

Deadlines are distinct from tasks.

A task represents work.

A deadline represents a date/time by which something must occur.

Example:

Task:

Prepare response

Deadline:

Response must be submitted by

15.09.2026 16:00

44. Deadline Model

A deadline should contain:

deadline_id

case_id

title

description

due_at

timezone

type

status
source
responsible_user
created_at
completed_at

45. Deadline Types

Possible values:

COURT
STATUTORY
CONTRACTUAL
INTERNAL
CLIENT
ADMINISTRATIVE
OTHER

The type should not automatically imply a particular legal calculation unless explicitly configured.

46. Date and Time Handling

All timestamps shall follow the conventions from v1.4:

- RFC3339/ISO 8601;
- timezone-aware;
- UTC storage where appropriate;
- explicit local timezone for user-facing legal deadlines.

Date-only legal dates must remain date-only.

47. Iceland Time Zone

For Icelandic legal workflows, the canonical local timezone should be:

Atlantic/Reykjavik

The application must not assume a generic fixed UTC offset.

The timezone database should be used for date/time calculations.

48. Deadline Calculation

Some deadlines may be calculated from an event.

Example:

Court event:

01.09.2026

Configured period:

14 days

Calculated deadline:

15.09.2026

The system must record the calculation inputs.

49. Deadline Calculation Transparency

A calculated deadline should expose:

trigger event

start date

calculation rule

calendar

result

Example:

Start:

01.09.2026

Rule:

+14 calendar days

Result:

15.09.2026

Users must be able to understand how a date was produced.

50. Legal Deadline Rules

Legal deadline calculations can be complex.

Rules may depend on:

- calendar days;
- working days;
- weekends;

- holidays;
- court rules;
- triggering events;
- receipt dates;
- local procedural rules.

Therefore deadline calculation must be implemented as configurable rules rather than scattered date arithmetic.

51. Legal Calendar

ILCMS should support configurable calendars.

Example:

```
Icelandic public holidays
Court-specific working days
Organization working days
Custom closure days
The calendar engine should be independently testable.
```

52. Deadline Override

Authorized users may manually override a calculated deadline.

An override must require:

```
new_due_date
reason
user
timestamp
The original calculated value must remain in history.
```

53. Deadline Changes

Changes should be auditable.

Example:

```
10.09:
Deadline = 15.09
```

12.09:

Court grants extension

New deadline = 22.09

The history must preserve both values.

54. Deadline Status

Recommended:

UPCOMING

DUE_SOON

OVERDUE

COMPLETED

WAIVED

CANCELLED

55. Deadline Escalation

The system should generate reminders based on configurable thresholds.

Example:

```
14 days before -> reminder
7 days before  -> reminder
2 days before  -> high priority
1 day before   -> urgent
overdue        -> escalation
```

56. Escalation

Escalation may notify:

- responsible lawyer;
- case manager;
- team leader;
- organization administrator.

Escalation rules must be configurable.

57. Deadline Notifications

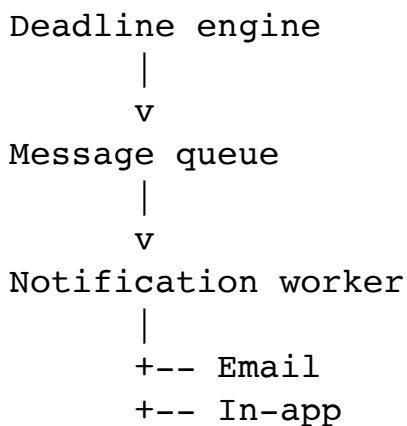
Notifications may be delivered through:

- in-app notifications;
- email;
- optional mobile/push notification;
- task dashboard.

Email should not be the only notification channel.

58. Notification Reliability

Notification delivery must be asynchronous.



Failed notifications should be retried.

59. Notification Audit

The system should record:

```
notification_id
recipient
type
created_at
sent_at
delivery_status
```

The content of sensitive notifications should be minimized.

60. Hearings

Cases should support hearings and appointments.

A hearing may contain:

hearing_id
case_id
type
court
location
start_at
end_at
status
description

61. Hearing Types

Examples:

HEARING
TRIAL
CASE_MANAGEMENT
MEDIATION
SETTLEMENT
INTERVIEW
MEETING
OTHER

62. Court Information

Court-related information should support:

court_name
court_case_number
jurisdiction
court_contact
submission_method

The model should allow multiple proceedings where a single matter moves through different stages.

63. Proceedings

A case may contain multiple legal proceedings.

Example:

Case

```
|
+-- District Court proceeding
|
+-- Appeal
|
+-- Supreme Court proceeding
```

Each proceeding should have its own:

- court;
- case number;
- status;
- dates;
- submissions;
- hearings.

64. Proceeding Status

Possible values:

```
PLANNED
ACTIVE
SUSPENDED
COMPLETED
APPEALED
CLOSED
```

65. Court Events

Court events should be represented explicitly.

Examples:

```
Filing
Hearing
Order
Decision
Deadline
Service
Settlement
Judgment
Appeal
```

Court events may automatically create tasks or deadlines.

66. Court Document Integration

Court events should be linkable to documents.

Example:

```
Court Decision
|
+-- Document
+-- Decision date
+-- Case
```

67. Correspondence

ILCMS should support case correspondence.

Examples:

- incoming letter;
- outgoing letter;
- email;
- phone note;
- portal message.

Correspondence should be linked to:

- case;
- party;
- user;
- documents.

68. Communication Record

A communication record may contain:

```
communication_id
case_id
type
direction
date
```

participants
subject
summary
document_id
created_by

The system should avoid duplicating full document contents in communication records.

69. Communication Types

EMAIL
PHONE
LETTER
MEETING
VIDEO_CALL
COURT_PORTAL
OTHER

70. Direction

INBOUND
OUTBOUND
INTERNAL

71. Notes

Users may create internal case notes.

Notes should have:

note_id
case_id
author
created_at
updated_at
content
visibility

72. Note Visibility

Notes may be:

CASE_TEAM
PRIVATE

ORGANIZATION

Private notes must never become visible merely because the user has case access.

73. Note Audit

Changes to sensitive notes should be audited.

If note editing is supported, historical versions may be retained according to policy.

74. Case Timeline

Each case should provide a chronological timeline.

Example:

```
02.09.2026 Case opened
02.09.2026 Client document received
03.09.2026 Lawyer assigned
05.09.2026 Court filing created
06.09.2026 Deadline created
10.09.2026 Hearing scheduled
```

75. Timeline Events

Events should reference domain objects rather than duplicate them.

Examples:

```
CASE_CREATED
USER_ASSIGNED
DOCUMENT_REGISTERED
TASK_CREATED
TASK_COMPLETED
DEADLINE_CREATED
DEADLINE_CHANGED
HEARING_SCHEDULED
COURT_DOCUMENT_SUBMITTED
CASE_STATUS_CHANGED
```

76. Timeline Ordering

Events should be ordered by:

1. event timestamp;
2. sequence number for identical timestamps.

The system must not depend solely on database insertion order.

77. Case Dashboard

The case dashboard should provide:

Case summary

- |
- +-- Status
- +-- Owner
- +-- Parties
- +-- Upcoming deadlines
- +-- Tasks
- +-- Hearings
- +-- Recent documents
- +-- Recent activity
- +-- Alerts

78. Work Queue

Users should have a personal work queue.

Example:

My Work

Overdue tasks	3
Deadlines this week	5
Unread documents	7
Upcoming hearings	2
Pending approvals	4

79. Team Dashboard

Team leaders may have:

- Team workload
- Open cases
- Overdue tasks
- Upcoming deadlines
- Unassigned cases

Cases without active owner
Access must respect organization permissions.

80. Workload Management

The system should provide workload indicators.

Example:

Lawyer A

25 active cases
8 deadlines this week
3 overdue tasks

Lawyer B

12 active cases
2 deadlines this week

These are operational indicators, not measures of legal quality.

81. Case Alerts

Cases may display alerts such as:

DEADLINE_SOON
OVERDUE_TASK
MISSING_REQUIRED_DOCUMENT
UNASSIGNED
CONFLICT_REVIEW_REQUIRED
LEGAL_HOLD
PROCESSING_FAILURE
COURT_EVENT_APPROACHING

Alerts should be actionable.

82. Required Case Information

Workflow templates may define mandatory information.

Example:

Before ACTIVE:

Client required
Responsible lawyer required

Classification required
Conflict check completed
The system should validate these conditions.

83. Case Closure

Closing a case should be a controlled workflow.

Example:

```
ACTIVE
  ↓
RESOLVED
  ↓
CLOSURE_REVIEW
  ↓
CLOSED
```

84. Closure Checklist

The system may require:

No unresolved mandatory tasks
No open critical deadlines
Required documents registered
Required metadata completed
Billing completed, if enabled
Client communication completed, if required
Retention policy assigned
Legal hold evaluated

85. Reopening a Case

Authorized users may reopen a closed case.

Reopening requires:

```
reason
user
timestamp
```

The case history must preserve the original closure.

86. Case Archiving

After closure and retention processing:

CLOSED

↓

RETENTION

↓

ARCHIVE

The archival process is governed by v1.6.

87. Case Templates

Organizations may define case templates.

A template may automatically provide:

Case type

Classification

Workflow

Default tasks

Default deadlines

Required metadata

Default team

Document folders/categories

88. Workflow Automation

Automation may create operational actions.

Example:

Event:

Case opened

Automation:

Create conflict review task

Create initial assessment task

Notify case owner

Automation must not silently perform legal decisions.

89. Automation Rules

Rules should have:

trigger

condition
action
enabled
version
created_by

Example:

TRIGGER:
Case status changed to ACTIVE

CONDITION:
case_type = CIVIL

ACTION:
Create task "Prepare initial case assessment"

90. Automation Safety

Automations must be:

- deterministic;
- auditable;
- idempotent;
- disableable;
- versioned.

An automation failure must not corrupt case data.

91. Scheduled Jobs

The scheduler from v1.1 should process:

- upcoming deadlines;
- overdue deadlines;
- recurring tasks;
- notifications;
- workflow timers;
- retention checks;

- stale cases;
- reconciliation.

92. Idempotent Scheduling

Scheduled jobs must tolerate repeated execution.

For example:

`Generate deadline reminder`

must not send the same reminder 100 times because a worker restarted.

A unique event/action identifier should be used.

93. Case Search

Users should be able to search by:

- case number;
- title;
- client;
- opposing party;
- court case number;
- classification;
- status;
- owner;
- date;
- tags.

Results must respect authorization.

94. Case Tags

Optional tags may provide flexible organization.

Examples:

URGENT
SETTLEMENT
INSURANCE
PROPERTY
CLIENT_PRIORITY

Tags should not replace formal classification.

95. Case Custom Fields

Organizations may need custom metadata.

ILCMS should support controlled custom fields.

Example:

```
custom_fields:  
    insurance_claim_number  
    internal_reference  
    business_unit
```

Custom fields must be typed and validated.

96. Time Tracking

Optional time tracking may be supported.

A time entry may contain:

```
time_entry_id  
case_id  
user_id  
start  
end  
duration  
description  
billable
```

Time tracking should remain separate from task completion.

97. Expenses

Optional case expenses may contain:

```
expense_id  
case_id
```

date
category
amount
currency
description
document_id
Currency should use ISO 4217 codes.

98. Billing Integration

Billing should be implemented as a separate module/integration rather than embedding accounting logic into the case domain.

Potential integration:

```
ILCMS
|
v
Billing Adapter
|
v
Accounting System
```

99. Legal Process Model

The system should distinguish:

```
Case
|
+-- Proceeding
    |
    +-- Court
    +-- Court case number
    +-- Events
    +-- Submissions
    +-- Hearings
    +-- Decisions
```

This allows a single matter to involve multiple procedural stages.

100. Appeals

An appeal should normally be represented as a new proceeding associated with the original case.

Example:

Case

```
|
+-- District Court Proceeding
|
+-- Appeal Proceeding
|
+-- Supreme Court Proceeding
```

The relationship must be explicit.

101. Decisions and Judgments

Court decisions should be represented as events/documents.

Example:

Court Decision Event

```
|
+-- decision_date
+-- court
+-- proceeding
+-- document_id
```

The document itself remains managed under v1.6.

102. Legal Deadlines and AI

If AI is later used to identify possible deadlines from documents:

Document

↓

AI extraction

↓

Potential deadline

↓

Human review

↓

Confirmed deadline

AI-generated deadlines must not automatically become authoritative legal deadlines without configurable human validation.

103. Deadline Confidence

AI-assisted deadline extraction may include:

confidence
source_document
source_page
extracted_text
model
model_version

The user must be able to verify the source.

104. Human Approval

High-impact workflow actions should support explicit human approval.

Examples:

AI identifies deadline

↓

Lawyer reviews

↓

Confirm deadline

↓

Official deadline created

This is especially important for legal deadlines.

105. Case Workflow Audit

The audit system shall record:

Case created
Case assigned
Case status changed
Workflow changed
Task created/completed
Deadline created/changed/completed
Hearing created/changed
Proceeding created
Court event recorded
Case closed/reopened

106. Workflow Security

Workflow definitions should be protected from unauthorized modification.

Changing a workflow can affect many active cases.

Therefore:

- workflow editing requires administrative permission;
- changes are audited;
- versions are immutable;
- active cases retain their workflow version unless migrated.

107. Business Rules Engine

A lightweight business-rules mechanism may be implemented.

It should support:

WHEN

IF

THEN

but avoid becoming a general-purpose programming language.

Complex legal logic should not be embedded into arbitrary user-defined scripts.

108. Case Event Architecture

Domain events should be published through the event architecture.

Example:

CaseStatusChanged

|

+-- Audit

+-- Notifications

+-- Workflow engine

+-- Search indexing

+-- Integrations

109. Event Ordering

Where workflow depends on event order, the system should provide:

- event IDs;
- timestamps;

- sequence numbers;
- correlation IDs.

Consumers must be able to handle duplicate events.

110. Case Import

Legacy case data may be imported.

Import should support:

- cases;
- parties;
- documents;
- assignments;
- historical events;
- deadlines;
- classifications.

Imported historical events should be marked as imported rather than pretending they happened inside ILCMS.

111. Historical Data

Example:

```
source_system = legacy_ilcms
source_id = 12345
imported_at = ...
This preserves provenance.
```

112. Case Deletion

Legal case systems should generally avoid normal physical deletion of cases.

Instead, cases should move through configured lifecycle states.

Where deletion is legally/operationally permitted, it must be:

- explicitly authorized;

- audited;
- retention-aware;
- legal-hold-aware.

113. Case Merge

Case merging should be treated as a controlled operation.

If two cases are merged:

Case A

```

  \
   → Master Case
  /

```

Case B

The historical relationship must remain available.

Documents, tasks and events must retain provenance.

114. Case Split

A case may sometimes need to be split.

The system should preserve:

Original Case

```

  |
  +-- Child Case A
  +-- Child Case B

```

Documents/events moved to the new case must remain traceable.

115. Case Relationships

Cases may be related:

```

RELATED_TO
PARENT_OF
CHILD_OF
DUPLICATE_OF
MERGED_FROM
SPLIT_FROM
APPEAL_OF

```

Relationships should be explicit.

116. Case Dashboard Performance

The case dashboard must not execute dozens of independent database queries.

The backend should provide an optimized case-summary endpoint.

Example:

`GET /api/v1/cases/{id}/summary`

The response may include:

- case metadata;
- parties;
- assignments;
- upcoming deadlines;
- open tasks;
- recent documents;
- recent events.

117. Pagination

Large collections must be paginated.

Examples:

`Documents`

`Tasks`

`Events`

`Communications`

`Audit events`

The API follows the pagination conventions defined in v1.4.

118. Concurrency

Concurrent edits must be handled safely.

Example:

Lawyer A edits case

Lawyer B edits same case

The system should detect stale versions rather than silently overwriting data.

119. Transaction Boundaries

Case mutations should be transactional.

For example:

Assign case

|

+-- Update assignment

+-- Create audit event

+-- Publish outbox event

These operations should remain consistent.

120. Outbox Pattern

The transactional outbox pattern from the backend architecture should be used for important events.

Example:

Database Transaction

|

+-- Case update

+-- Audit event

+-- Outbox event

|

COMMIT

|

v

Message Worker

This avoids losing events when the database succeeds but message publication fails.

121. Notifications and Tasks

Notifications should never be the authoritative source of task/deadline state.

The database remains authoritative.

Example:

Deadline = 15.09

Notification = failed
The deadline still exists.

122. User Calendar Integration

Future versions may integrate with:

- Outlook;
- Google Calendar;
- CalDAV;
- internal calendars.

Calendar integration should expose only authorized case information.

123. Calendar Security

External calendar events should avoid unnecessary confidential information.

Instead of:

"Confidential Case: Jón Jónsson v. Company X"
an external calendar may use:

"ILCMS Legal Deadline"
with details accessible through ILCMS.

The exact behavior should be configurable.

124. Mobile / Responsive Use

The core workflow should work on responsive web interfaces.

High-priority mobile functions:

- view case;
- view tasks;
- complete task;
- view deadline;
- acknowledge notification;

- view documents;
- record communication.

Complex document editing may remain desktop-oriented.

125. Offline Considerations

Core legal operations should require connectivity.

Offline operation must not allow uncontrolled modifications that later bypass authorization or conflict controls.

If offline functionality is introduced later, it requires a dedicated synchronization/security design.

126. Reporting

The system should support operational reporting:

Cases by status
Cases by lawyer
Upcoming deadlines
Overdue tasks
Cases by classification
Case aging
Workload
Court proceedings
Reports must obey authorization.

127. Case Aging

Case age may be calculated from:

current date – case opening date
but should also support:

active duration
pending duration
where workflow history permits.

128. SLA Support

Organizations may define internal service-level targets.

Example:

New inquiry

↓

Initial response within 2 business days
SLA tracking must be clearly distinguished from legal deadlines.

129. SLA Escalation

SLA breaches may trigger:

warning

team notification

manager escalation

They must not be confused with statutory/court deadlines.

130. Case Closure Analytics

Reports may identify:

- average case duration;
- number of reopened cases;
- overdue tasks at closure;
- unresolved items;
- cases awaiting closure.

These are management metrics and should not be presented as legal-performance judgments without appropriate interpretation.

131. Data Model

The v1.3 database should support at least:

cases

case_status_history

case_assignments

case_access

case_relationships

case_parties

parties

workflows
workflow_versions
workflow_states
workflow_transitions

tasks
task_dependencies
task_templates

deadlines
deadline_history
deadline_rules
calendars

hearings
proceedings
court_events

communications
notes

case_events
notifications

time_entries
expenses

132. Suggested Case Table

Conceptually:

cases

id
organization_id
case_number
title
case_type
classification_id
status
priority
confidentiality_level
opened_at
closed_at

owner_user_id
workflow_id
workflow_version_id
created_by
created_at
updated_at

133. Suggested Assignment Table

case_assignments

id
case_id
user_id
team_id
role
is_primary
assigned_at
assigned_by
removed_at
removed_by
Historical records should remain available.

134. Suggested Task Table

tasks

id
case_id
title
description
status
priority
assigned_user_id
assigned_team_id
due_at
created_at
created_by
completed_at
completed_by

135. Suggested Deadline Table

```
deadlines
-----
id
case_id
title
type
due_at
timezone
source
calculation_rule
status
responsible_user_id
created_at
created_by
completed_at
```

136. Suggested Court Proceeding Model

```
proceedings
-----
id
case_id
court_id
court_case_number
type
status
started_at
ended_at
court_events
-----
id
proceeding_id
event_type
event_at
description
document_id
created_by
```

137. API Endpoints

The v1.4 REST API should support at least:

```
POST    /api/v1/cases
GET     /api/v1/cases
```

GET /api/v1/cases/{id}
PATCH /api/v1/cases/{id}

POST /api/v1/cases/{id}/assign
POST /api/v1/cases/{id}/close
POST /api/v1/cases/{id}/reopen

GET /api/v1/cases/{id}/summary
GET /api/v1/cases/{id}/timeline

GET /api/v1/cases/{id}/tasks
POST /api/v1/cases/{id}/tasks

GET /api/v1/tasks
PATCH /api/v1/tasks/{id}
POST /api/v1/tasks/{id}/complete

GET /api/v1/deadlines
POST /api/v1/deadlines
PATCH /api/v1/deadlines/{id}
POST /api/v1/deadlines/{id}/complete

GET /api/v1/cases/{id}/hearings
POST /api/v1/cases/{id}/hearings

GET /api/v1/cases/{id}/proceedings
POST /api/v1/cases/{id}/proceedings

GET /api/v1/cases/{id}/communications
POST /api/v1/cases/{id}/communications

GET /api/v1/cases/{id}/notes
POST /api/v1/cases/{id}/notes

Exact request/response schemas remain defined in the OpenAPI specification.

138. Workflow APIs

Recommended:

GET /api/v1/workflows
POST /api/v1/workflows

GET /api/v1/workflows/{id}
POST /api/v1/workflows/{id}/versions

GET /api/v1/workflows/{id}/states
GET /api/v1/workflows/{id}/transitions
Workflow administration requires elevated authorization.

139. Deadline APIs

Recommended:

GET /api/v1/deadlines/upcoming
GET /api/v1/deadlines/overdue
These endpoints should support filtering by:

- organization;
- user;
- team;
- case;
- date range;
- deadline type.

140. Calendar APIs

Potential future endpoints:

GET /api/v1/calendar
GET /api/v1/calendar/deadlines
GET /api/v1/calendar/hearings
The response must contain only authorized events.

141. Background Jobs

The v1.1 job infrastructure should handle:

deadline reminders
SLA checks
workflow timers
recurring tasks
notification delivery
calendar synchronization

case aging calculations

Jobs should be observable through the job system defined in v1.4.

142. Audit Integration

All high-impact operations must create audit events.

Examples:

CaseCreated

CaseAssigned

CaseAccessChanged

CaseStatusChanged

CaseClosed

CaseReopened

TaskCreated

TaskCompleted

TaskReopened

DeadlineCreated

DeadlineChanged

DeadlineCompleted

DeadlineOverridden

WorkflowChanged

143. Authorization Integration

Every case operation must use the v1.5 authorization system.

The effective authorization may depend on:

Organization

+

Role

+

Case assignment

+

Case access

+

Document access

+

Confidentiality

144. Workflow Security Boundary

Workflow automation must not be able to bypass authorization.

For example:

Workflow:

Create document

↓

Document authorization

↓

Normal authorization rules

Automation is not an administrator.

145. Case Event Security

Events must contain only the information required by consumers.

Do not publish full confidential case contents through the message broker.

Prefer:

```
{
  "event": "DeadlineCreated",
  "case_id": "uuid",
  "deadline_id": "uuid",
  "organization_id": "uuid"
}
```

146. Legal Deadline Safety Principle

ILCMS shall follow this rule:

The system may calculate, track and remind about deadlines, but the responsible legal professional remains responsible for verifying the applicable legal deadline.

This is particularly important where:

- procedural rules differ;
- holidays affect calculations;
- court-specific rules apply;
- extensions are granted;

- service/receipt dates are disputed.

147. Workflow Safety Principle

Similarly:

Workflow automation may automate administrative processes but must not silently make legal judgments.

148. Icelandic Localization

The workflow system shall support Icelandic terminology.

Examples:

Case	Mál
Document	Skjal
Deadline	Frestur
Task	Verkefni
Hearing	Þinghald / málflutningur, as configured
Party	Aðili
Client	Skjólstaðingur / umbjóðandi, depending on
context	
Court	Dómstóll
Decision	Úrskurður / dómur, depending on document
type	

Terminology should be configurable because precise terminology depends on legal context.

149. Language Architecture

Machine-readable values remain stable:

CASE_STATUS_ACTIVE

DEADLINE_OVERDUE

TASK_COMPLETED

The frontend translates them into Icelandic.

This prevents language changes from breaking APIs.

150. Accessibility

The case-management interface should support accessibility standards appropriate for a professional web application.

Particular attention should be given to:

- keyboard navigation;
- screen readers;
- high information density;
- deadline status indicators;
- accessible forms;
- accessible tables;
- clear error messages.

Color must not be the only indicator of urgency.

151. Performance

Case dashboards should load quickly even for large cases.

The API should avoid:

- N+1 queries;
- loading all documents;
- loading complete audit history;
- unbounded event lists.

Large collections must be paginated.

152. Caching

Caching may be used for:

- workflow definitions;
- classification trees;
- organization configuration.

Do not cache authorization-sensitive case data without carefully scoped keys and invalidation.

153. Testing

Testing shall cover:

Case lifecycle

Create
Open
Activate
Resolve
Close
Reopen
Archive

Tasks

Create
Assign
Start
Complete
Reopen
Cancel

Deadlines

Create
Calculate
Modify
Complete
Overdue
Escalate

Workflow

Transition
Invalid transition
Authorization
Versioning
Automation

154. Deadline Test Suite

The deadline engine requires extensive tests covering:

- normal calendar days;
- weekends;

- Icelandic holidays;
- timezone boundaries;
- leap years;
- year boundaries;
- manual overrides;
- extensions;
- duplicate processing;
- daylight/timezone changes if relevant;
- historical rules.

155. Workflow Integration Tests

Example:

Create civil case



Workflow assigned



Initial tasks generated



Task completed



Next workflow state



Deadline generated



Reminder scheduled

The entire chain should be tested end-to-end.

156. Security Tests

Test:

Unauthorized user

Wrong organization

Wrong team

Wrong case

Read-only user

Case owner
Organization admin
System admin
Suspended user
Deactivated user
No workflow action should bypass v1.5 authorization.

157. Acceptance Criteria

v1.7 is complete when:

1. Cases can be created and managed.
2. Case numbers are generated uniquely.
3. Case lifecycle states are implemented.
4. Case assignments are supported.
5. Assignment history is preserved.
6. Parties can be associated with cases.
7. Conflict-check workflows are supported.
8. Configurable workflows exist.
9. Workflow versions are preserved.
10. Tasks can be created, assigned and completed.
11. Task dependencies work.
12. Deadlines can be created and tracked.
13. Deadline history is preserved.
14. Deadline calculations are transparent.
15. Icelandic calendar configuration is supported.
16. Deadline reminders work asynchronously.
17. Escalations work.
18. Hearings can be scheduled.
19. Proceedings can be represented.
20. Court events can be recorded.
21. Correspondence can be recorded.

22. Internal notes are supported.
23. Case timelines are available.
24. Case dashboards are available.
25. User/team work queues are available.
26. Cases can be closed and reopened securely.
27. Case relationships are supported.
28. Search respects authorization.
29. Workflow automation is auditable.
30. All high-impact operations generate audit events.
31. Database transactions maintain consistency.
32. Background processing is idempotent.
33. Legal deadline calculations are configurable and human-verifiable.
34. AI-generated deadlines, if introduced, require human confirmation.
35. Icelandic localization is supported.
36. The entire workflow is compatible with the security architecture in v1.5.
37. The case/document/records lifecycle integrates correctly with v1.6.

158. MVP Implementation Priority

Phase 1 — Core Case Management

Implement:

Cases

Parties

Assignments

Case status

Case classification

Case access

Case dashboard

Case timeline

Phase 2 — Work Management

Implement:

Tasks
Task dependencies
Deadlines
Notifications
Work queues
Hearings

Phase 3 — Workflow

Implement:

Workflow templates
Workflow states
Transitions
Automation
SLA tracking
Escalations

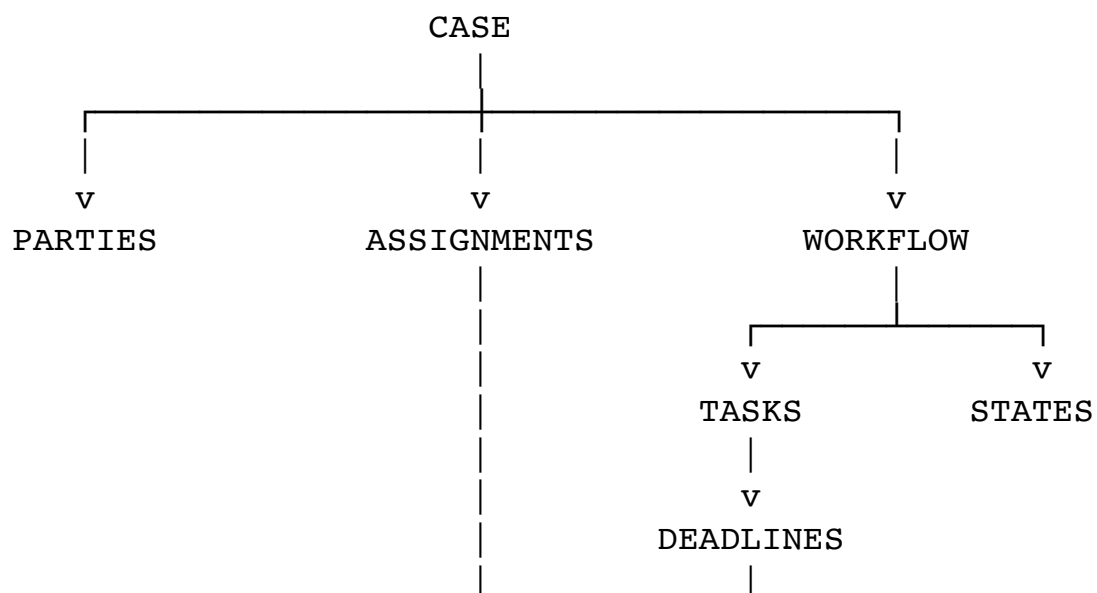
Phase 4 — Advanced Legal Process

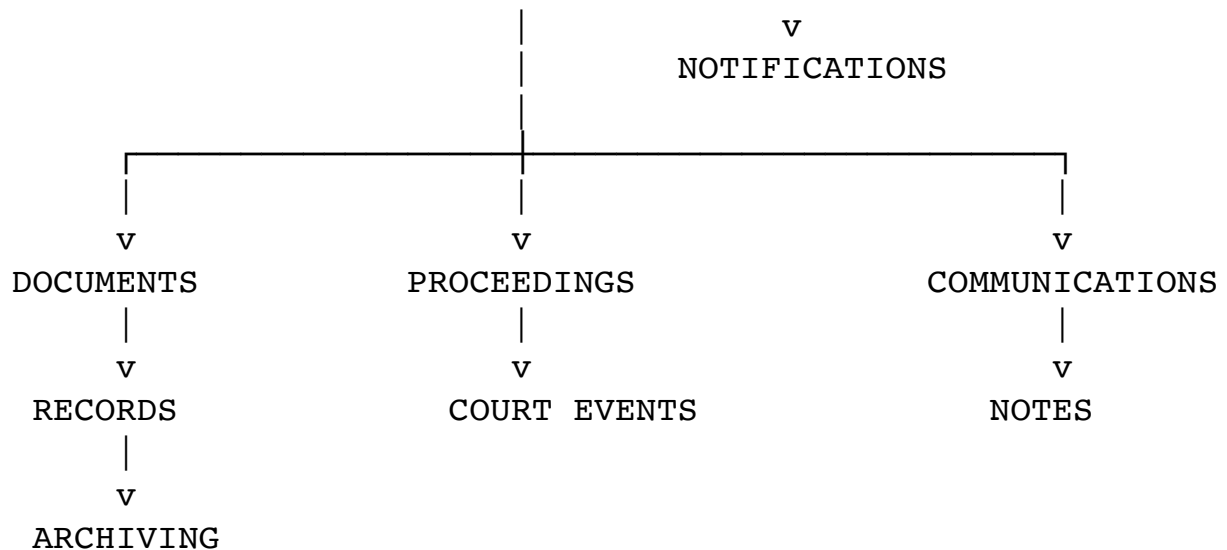
Implement:

Proceedings
Court events
Court submission workflows
Advanced deadline rules
Calendar integration
AI-assisted deadline identification

159. Overall Case Architecture

The resulting operational model is:





160. Final Architectural Principle

The fundamental principle of v1.7 is:

A case is the central operational context that connects people, documents, tasks, deadlines, communications and legal proceedings, while workflow automation coordinates the work without replacing human legal judgment.

The resulting ILCMS architecture now has:

- v1.1 Infrastructure
- v1.2 Backend
- v1.3 Database
- v1.4 REST API
- v1.5 Security
- v1.6 Documents & Records
- v1.7 Case Management & Workflow

This establishes the core application platform.

The next major specification should address the **user-facing experience and operational collaboration layer around these capabilities**, including advanced search, dashboards, notifications, calendars, reporting, and possibly external integrations.

The next logical step would be **v1.8 — Search, Notifications, Calendars, Reporting & Collaboration Specification**. That would connect the underlying case/document/workflow engine to the day-to-day information experience for lawyers and case managers.